

Q. Why Java 8 ?

- ① Concise and minimal code.
- ② To utilize functional programming benefits in Java, Java 8 came.
- ③ To enable Parallel programming.

Features of Java 8 :

- ① Lambda Expression
- ② Stream API
- ③ Date and Time API
- ④ Base 64 Encode and Decode
- ⑤ Method reference and constructor reference.
- ⑥ Default methods in Interface.
- ⑦ Static methods in Interface.
- ⑧ Functional Interface
- ⑨ Optional class
- ⑩ Java IO Improvements
- ⑪ Collection API Improvements.

λ Lambda Expression :

→ Lambda expression is an anonymous function.

i.e

1. Not having any name
2. Not having any return type
3. Not having modifiers.

Steps to make any function lambda expression :

1. Remove modifiers
2. Remove return type
3. Remove method name
4. Place Arrow

Example 1:

Normal Function:

```
private void SayHello () {
    System.out.println("Hello world!");
}
```

Lambda Expression:

```
() -> { System.out.println("Hello world!"); }
```

Example 2:

Normal Function:

```
private void add (int a, int b) {
    System.out.println(a+b);
}
```

Lambda Expression:

```
(int a, int b) -> { System.out.println(a+b); }
```

Example 3:

Normal Function:

```
private int getStringLength (String str) {
    return str.length();
}
```

Lambda Expression:

```
(String str) -> { return str.length(); }
```

Properties of Lambda Expression:

- ① If body has just one statement then we can remove curly braces.

Example: `() -> System.out.println("Hello world!");`

- ② Use type inference, compiler guess the situation or context.

Example:

```
private void add (int a, int b) {
    System.out.println(a+b);
}
```



```
(int a, int b) -> { System.out.println(a+b); }
```



```
(a, b) -> System.out.println(a+b);
```


③ No return keyword

```
private int getStringLength(String str) {  
    return str.length();  
}
```

↓

```
(String str) → { return str.length(); }
```

↓

```
(str) → str.length();
```

④ If only one Parameter remove small brackets

```
(str) → str.length();
```

```
str → str.length();
```

Benefits:

- ① To enable functional programming in Java
- ② To make code more readable, maintainable and concise code.
- ③ To enable Parallel processing
- ④ JAR file size reduction
- ⑤ Elimination of shadow variables

Functional Interface:

→ Interface having exactly single abstract method but can have any numbers of default and static methods, is called Functional Interface.

→ We can invoke lambda expression by using functional interface.

NOTE:

When we want to create a Functional Interface we need to write @FunctionalInterface annotation before interface declaration.

@FunctionalInterface

```
public interface MyInterface {  
    public void SayHello();  
    default void SayBye();  
};  
public static void SayOK() {  
};  
}
```

Inheritance in Functional Interface:

If an Interface extends to any Functional Interface then it will also become Functional Interface.

Case 1: ✓

```
@FunctionalInterface  
public interface Child extends Parent {  
}
```

Case 2: ✗

```
@FunctionalInterface  
public interface Child extends Parent {  
    public void SayBye();  
}
```

Case 3: ✓

```
@FunctionalInterface  
public interface Child extends Parent {  
    public void SayHello();  
}
```

Case 5: ✓

```
@FunctionalInterface  
public interface Child extends Parent {  
    public void SayHello();  
    default void SayBye();  
};  
}
```

Case 6: ✓

```
@FunctionalInterface  
public interface Child extends Parent {  
    public void SayHello();  
    public static void SayOK();  
};  
}
```


Default Methods Inside Interface:

- Until 1.7 only Public abstract methods were allowed whether we declare by writing or not.
- Similarly Public static final variable were allowed.
- Since Java 8 we can have concrete methods as well inside interface.

Default Method:

Q. What?

→ A default method is a method with a body inside an interface.

Q. Why?

→ To add new methods to an interface without breaking old classes that already implement it.

Key points:

- ① Uses the keyword default.
- ② Has a method body.
- ③ Can be overridden by implementation class.
- ④ Called using object of the class.

Example:

```
interface Animal {
    default void Sound() {
        System.out.println("Animal Makes a Sound!");
    }
}
```

```
class Dog implements Animal {
    void run() {
        System.out.println("Dog is running");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.Sound();
        d.run();
    }
}
```

OUTPUT:

Animal Makes a Sound!
Dog is running.

Static Method Inside Interface:

Q. What?

→ A static method inside an interface belongs to the interface itself rather than objects of the implementation class.

Q. Why?

→ To keep utility/helper methods related to the interface in one place.

Key points:

- ① Uses the keyword static.
- ② Has a method body.
- ③ Cannot be overridden, because the static method of interface will not be visible inside implementation.
- ④ Called using interface name, not the object of implementation class.

Example 1:

```
interface MathUtils {
    static void add (int a, int b ) {
        System.out.println( a+b );
    }
}
```

```
public class Main {
    public static void main (String[] args) {
        MathUtils.add (10, 20);
    }
}
```

OUTPUT:
30

Example 2:

```
interface Parent {
    static void sayHello () {
        System.out.println ( "Parent Says Hello " );
    }
}
```

```
class Child implements Parent {
    static void sayHello () {
        System.out.println ( "Child Says Hello " );
    }
}
```

NOTE:

Here inside child class we are not overriding sayHello () method.
Because sayHello () method of Parent interface will not be visible inside child class.
So child class consider it as new.

Use of Functional Interface & Lambda Expression:

240

```
public interface Employee {
    String getName();
}

public class SoftwareEngineer implements Employee {
    @Override
    public String getName() {
        return "Software Engineer";
    }
}

public class Main {
    public static void main(String[] args) {
        Employee employee = new SoftwareEngineer();
        System.out.println(employee.getName()); // Software Engineer
    }
}
```

Problem:

For a small task we need to create implementation class.

```
public interface Employee {
    String getName();
}

public class Main {
    public static void main(String[] args) {
        Employee employee = () -> "Software Engineer";
        System.out.println(employee.getName());
    }
}
```

Solution:

Using Lambda expression we can ignore the separate implementation class.

** Lambda expression will act as the implementation for the single abstract method.

Create Thread Using Lambda Expression:

241

```
public class Main {  
    public static void main(String[] args) {  
        Runnable runnable = () -> {  
            @Override  
            public void run() {  
                for (int i = 1; i <= 5; i++) {  
                    System.out.println(i);  
                }  
            }  
        };  
        Thread t = new Thread(runnable);  
        t.start();  
    }  
}
```

OUTPUT:

1
2
3
4
5

Comparator Using Lambda Expression:

Example 1:

```
public class SortListInDescendingOrder {  
    public static void main(String[] args) {  
        List<Integer> list = Arrays.asList(2, 6, 0, 99, 6);  
        Collections.sort(list, (a, b) -> b - a);  
        System.out.println(list); // [99, 6, 6, 2, 0]  
    }  
}
```

Example 2:

```
public class SortSetInDescendingOrder {  
    public static void main(String[] args) {  
        Set<Integer> S = new TreeSet<>();  
        S.add(22);  
        S.add(1);  
        S.add(13);  
        System.out.println("Before Manual Sorting: " + S); // Before Manual Sorting: [1, 13, 22]  
        Set<Integer> SS = new TreeSet<>((a, b) -> b - a);  
        SS.add(22);  
        SS.add(1);  
        SS.add(13);  
        System.out.println("After Manual Sorting: " + SS); // After Manual Sorting: [22, 13, 1]  
    }  
}
```


Example 3:

```

public class SortMapInDescendingOrder {
    public static void main (String[] args) {
        Map<Integer, String> m = new TreeMap<>();
        m.put (2, "z");
        m.put (3, "f");
        m.put (1, "y");
        System.out.println ("Before Manual Sorting: " + m); // Before Manual Sorting: {1=y, 2=z, 3=f}
        Map<Integer, String> sm = new TreeMap<>((a,b) -> b-a);
        m.put (2, "z");
        m.put (3, "f");
        m.put (1, "y");
        System.out.println ("After Manual Sorting: " + sm); // After Manual Sorting: {3=f, 2=z, 1=y}
    }
}

```

Example 4:

```

class Student {
    public private Integer id;
    public private String name;
    public Student (Integer id, String name) {
        this.id = id;
        this.name = name;
    }
    public String toString () {
        return this.id + ":" + this.name;
    }
}

```

```

public class Main {
    public static void main (String[] args) {
        Student s1 = new Student (2, "Vipul");
        Student s2 = new Student (3, "Raj");
        Student s3 = new Student (1, "Akshat");
        List<Student> list = Arrays.asList (s1, s2, s3);
        Collections.sort (list, (a,b) -> b.id - a.id);
        System.out.println (list); // {3: AKSHAT, 3: Raj, 2: Vipul}
    }
}

```

Anonymous Inner Class Vs Lambda Expression:

243

NOTE:

Lambda Expression works for only functional Interface which will have only one abstract method. In this case Lambda expression will act as the implementation for the abstract method.

But when interface will have more than one abstract method then we need to use Anonymous inner class.

Example:

```
Public class Employee {  
    String getSalary();  
    String getDesignation();  
}  
  
Public class Main {  
    Public static void main (String[] args) {  
        Employee employee = new Employee() {  
            @Override  
            Public String getSalary() {  
                return "10000";  
            }  
            @Override  
            Public String getDesignation() {  
                return "Software Engineers";  
            }  
        };  
  
        System.out.println ( employee.getSalary () ); // 10000  
        System.out.println ( employee.getDesignation () ); // Software Engineers  
    }  
}
```


Example 2:

```
public class Main {
    private void difference () {
        Employee employee1 = () -> {
            int x = 10; // Local variable
            return "10000";
        };
    }
}
```

```
public interface Employee {
    String getSalary ();
}
```

```
Employee employee2 = new Employee () {
    int x = 10; // Instance variable, since it is Anonymous class.
```

@Override

```
public String getSalary () {
    System.out.println("this: x");
    return "20000";
}
```

}

Predicate:

A predicate in Java is a functional interface which belongs to `java.util.function` package that represents boolean valued function.

Methods:

① `boolean test (T t);`

Example: Check number is Even or Not using predicate.

```

public class Main {
    public static void main (String[] args) {
        Predicate<Integer> predicate = x → x % 2 == 0;
        List<Integer> numbers = Arrays.asList (1, 2, 3, 4, 5);
        for (Integer i : numbers) {
            if (predicate.test (i)) {
                System.out.println (i + " IS Even Number");
            }
        }
    }
}

```

OUTPUT:

2 IS Even Number
4 IS Even Number

Example: Check wheather name is starting with 'A' or 'a' using predicate

```

public class Main {
    public static void main (String[] args) {
        Predicate<Integer> isEven = x → x % 2 == 0;
        Predicate<String> startsWithLetterA = x → x.toLowerCase().charAt(0) == 'a';
        System.out.println (startsWithLetterA.test ("Abhishek"));
    }
}

```

OUTPUT:

true

② default Predicate<T> and (Predicate<? super T> others) {
}

Example: Check wheather the name starts with 'A' or 'a' and ends with 'K' or 'k'.

```
public class And {
    public class static void main (String[] args) {
        Predicate<String> startsWithLetterA = x → x.toLowerCase().charAt(0) == 'a';
        Predicate<String> endsWithLetterK = x → x.toLowerCase().charAt(x.length()-1) == 'k';
        Predicate<String> and = startsWithLetterA.and(endsWithLetterK);
        System.out.println (and.test("Abhishek")); // true.
    }
}
```

③ default Predicate<T> or (Predicate<? super T> other) {
}

Example: Check wheather the name starts with 'A' or 'a' or ends with 'K' or 'k'.

```
public class OR {
    public static void main (String[] args) {
        Predicate<String> startsWithLetterA = x → x.toLowerCase().charAt(0) == 'a';
        Predicate<String> endsWithLetterK = x → x.toLowerCase().charAt(x.length()-1) == 'k';
        Predicate<String> or = startsWithLetterA.or(endsWithLetterK);
        System.out.println (or.test("AKash")); // true
    }
}
```

④ default Predicate<T> negate() {
}

Example:

```
public class Negate {
    public class static void main (String[] args) {
        Predicate<String> startsWithLetterA = x → x.toLowerCase().charAt(0) == 'a';
        System.out.println (startsWithLetterA.negate().test("Abhishek")); // false
    }
}
```

⑤ `Static <T> Predicate <T> isEqual (Object targetRef) {
}`

Example:

```
Public class Equal {
    Public Static void main (String[] args) {
        Predicate <Object> predicate = Predicate.isEqual ("Abhishek");
        System.out.println (predicate.test ("Abhishek") ); // true
    }
}
```

Function:

Function in Java is a functional interface which belongs to java.util.function package and it represents a function that takes in one argument and produces a result.

Methods:

① `R apply (T t);`

Example:

```
Public class Main {
    Public Static void main (String[] args) {
        Function <String, Integer> length = x → x.length();
        System.out.println (length.apply ("Abhishek")); // 8
    }
}
```

② `default <V> Function <T, V> andThen (Function <? Super R, ? extends V> after) {
}`

Example:

```
Public class AndThen {
    Public Static void main (String[] args) {
        Function <String, String> function1 = s → s.toUpperCase();
        Function <String, String> function2 = s → s.substring (0,3);
        System.out.println (function1.andThen(function2).apply ("VIP")); // VIP
    }
}
```


③ `default <V> Function<V,R> compose (Function<?, Super V, ? extends T> before) {
}`

Example:

```
Public class Compose {
    Public static void main (String[] args) {
        Function<Integer, Integer> function1 = x → 2 * x;
        Function<Integer, Integer> function2 = x → x * x * x;
        System.out.println (function1.andThen(function2).apply(3)); // 216
        System.out.println (function2.andThen(function1).apply(3)); // 54
        System.out.println (function1.compose(function2).apply(3)); // 54
    }
}
```

④ `Static <T> Function<T,T> identity() { return t → t; }`

Example:

```
Public class Identity {
    Public static void main (String[] args) {
        Function<String, String> identityFunction = Function.identity();
        System.out.println (identityFunction.apply ("Vipul")); // Vipul
    }
}
```

Consumer:

P49

Consumer in Java is a functional interface which belongs to java.util.function Package that represents a function which will accept one argument as input but not return any result.

Methods:

① `void accept(T t);`

Example:

```
Public class Main {  
    Public Static void main(String[] args) {  
        Consumer<String> consumer = s -> System.out.println(s);  
        Consumer<List<Integer>> listConsumer = li -> {  
            for (Integer i : li) {  
                System.out.println(i);  
            }  
        };  
        consumer.accept("Abhishek");  
        listConsumer.accept(Arrays.asList(1,2,3,4,5));  
    }  
}
```

OUTPUT:

Abhishek

1

2

3

4

5

② `default Consumer<T> andThen(Consumer<? super T> after) { }`

Example:

```
Public class Main {  
    Public Static void main(String[] args) {  
        Consumer<List<Integer>> listConsumer1 = li -> {  
            for (Integer i : li) {  
                System.out.println(i+100);  
            }  
        };  
        Consumer<List<Integer>> listConsumer2 = li -> {  
            for (Integer i : li) {  
                System.out.println(i);  
            }  
        };  
    }  
}
```

OUTPUT:

1

2

3

101

102

103

list Consumer2 . and Then (list Consumer1) . accept (Arrays.asList (1,2,3));

250

Supplier:

Supplier in Java is a functional interface which belongs to java.util.function package that represents a function that will not take any argument as input but produce a value.

Method:

① `T get();`

Example 1:

```
Public class Main {  
    Public Static void main (String[] args) {  
        Supplier < Integer > Supplier = () -> 1;  
        System.out.println (Supplier.get());  
    }  
}
```

OUTPUT:
1

Example 2:

```
Public class {  
    Public Static void main (String[] args) {  
        Predicate < Integer > predicate = x -> x % 2 == 0;  
        Consumer < Integer > consumer = x -> System.out.println (x);  
        Function < Integer, Integer > function = x -> x * x;  
        Supplier < Integer > Supplier = () -> 100;  
        if (predicate.test (Supplier.get())) {  
            consumer.accept (function.apply (Supplier.get()));  
        }  
    }  
}
```

OUTPUT:

10000

BiPredicate:

BiPredicate in Java is a functional interface which belongs to java.util.function package that represents boolean valued function that will take two arguments.

Methods:

Example:

```
public class Main {  
    public static void main (String[] args) {  
        BiPredicate<Integer, Integer> biPredicate = (x, y) → x%2==0 && y%2==0;  
        System.out.println (biPredicate.test (2, 4)); // true.  
    }  
}
```

BiFunction:

BiFunction in Java is a functional interface which belongs to java.util.function package that represents a function that will take two arguments and produce a result.

Example:

```
public class Main {  
    public static void main (String[] args) {  
        BiFunction<String, String, Integer> biFunction = (x, y) → x.length() + y.length();  
        System.out.println (biFunction.apply ("Hi", "Hello")); // 7  
    }  
}
```

BiConsumer:

BiConsumer in Java is a functional interface which belongs to java.util.function package that represents a function which will accept two arguments but not return any value.

Example:

```
public class Main {  
    public static void main (String[] args) {  
        BiConsumer<Integer, Integer> biConsumer = (x, y) → {  
            System.out.println (x+y);  
        };  
        biConsumer.accept (2, 4); // 6  
    }  
}
```


Unary Operators:

Unary Operator in Java is a functional interface which belongs to java.util.function package that represents a function which will take one argument and produce result of same type.

Example:

```

public class Main {
    public static void main (String[] args) {
        UnaryOperator<Integer> unary = x → x * x;
        System.out.println(unary.apply(5)); // 25
    }
}

```

Binary Operators:

Binary Operator in Java is a functional interface which belongs to java.util.function package that represents a function which will take two arguments of same type and produce a result of same type.

Example:

```

public class Main {
    public static void main (String[] args) {
        BinaryOperator<String> binary = (str1, str2) → str1 + str2;
        System.out.println(binary.apply("Hello", "World")); // Hello World.
    }
}

```

Method Reference:

Method references allow us to refer to a method without invoking it, making our code cleaner and more readable. They can be used in place of lambda expression when the lambda expression only calls an existing method.

Example 1:

```
public class Test {  
    public static void print (String s ) {  
        System.out.println (s);  
    }  
    public static void main (String[] args) {  
        List<String> students = Arrays.asList ("Alice", "Bob", "Charlie");  
        students.forEach (Test :: print);  
    }  
}
```

OUTPUT:

Alice
Bob
Charlie

Example 2:

```
public class Test {  
    public void print (String s ) {  
        System.out.println (s );  
    }  
    public static void main (String[] args) {  
        Test test = new Test ();  
        List<String> students = Arrays.asList ("Alice", "Bob", "Charlie");  
        students.forEach (test :: print);  
    }  
}
```

OUTPUT:

Alice
Bob
Charlie

Constructor Reference:

254

Constructor reference allow us to refer to a class's constructor without invoking it.

Example 1: Without Constructor Reference

```
public class Student {  
    private String name;  
  
    public Student(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        List<String> names = Arrays.asList("Alice", "Bob", "Charlie");  
        List<Student> students = names.stream().map(x -> new Student(x)).collect(  
            Collectors.toList());  
    }  
}
```

Example 2: With Constructor Reference

```
public class Test {  
    public static void main(String[] args) {  
        List<String> names = Arrays.asList("Alice", "Bob", "Charlie");  
        List<Student> students = names.stream().map(Student::new).collect(Collectors.toList());  
    }  
}
```

Java Stream API, introduced in Java 8, provides a modern, functional approach to process sequence of elements like collections or arrays.

Q. Why Stream?

- ① Readability: Streams provide a more readable and expressive way to perform operations on collections.
- ② Flexibility: Streams allow you to perform a variety of operations on collection such as filtering, mapping and reducing.
- ③ Parallelism: Streams can be processed in parallel, which can provide a significant performance boost for large collections.
- ④ Encapsulation: Streams encourage encapsulation, as you can perform a series of operation on a collection without modifying the original data.

Creating Stream

```
public class Test {  
    public static void main (String[] args) {  
        List<String> list = Arrays.asList("apple", "banana", "cherry");  
        Stream<String> myStream = list.stream();  
  
        String[] array = {"apple", "banana", "cherry"};  
        Stream<String> stream = Arrays.stream(array);  
  
        // Direct way of stream creation  
        Stream<Integer> integerStream = Stream.of(1, 2, 3);  
        Stream<Integer> limit = Stream.iterate(0, n -> n+1).limit(100);  
        Stream<String> limit1 = Stream.generate(() -> "hello").limit(5);  
    }  
}
```


Stream Operations:

- ① **filter()**: The **filter()** method is used to select elements as per the Predicate Passed as an argument.

Syntax: `Stream<T> filter(Predicate<? super T> predicate)`

Example: Filter Even numbers from List

```
public class Filter {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
        List<Integer> filteredList = list.stream().filter(x -> x % 2 == 0).collect(Collectors.toList());
        System.out.println("Filtered List: " + filteredList); // [2, 4, 6, 8, 10]
    }
}
```

- ② **map()**: The **map** method is used to return a stream consisting of the results applying the given function to the elements of this stream.

Syntax: `<R> Stream<R> map(Function<? super T, ? extends R> mapper)`

Example:

```
public class Map {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(2, 4, 6, 8, 10);
        List<Integer> mappedList = list.stream().map(x -> x / 2).collect(Collectors.toList());
        System.out.println("Mapped List: " + mappedList); // [1, 2, 3, 4, 5]
    }
}
```

- ③ **distinct()**: Removes duplicate elements.

Syntax: `Stream<T> distinct()`

Example:

```
public class Distinct {
    public static void main(String[] args) {
        List<Integer> list = Arrays.asList(2, 4, 6, 8, 6, 10, 10);
        List<Integer> distinctList = list.stream().distinct().collect(Collectors.toList());
        System.out.println("Distinct List: " + distinctList); // [2, 4, 6, 8, 10]
    }
}
```

④ Sorted(): The sorted method is used to sort the stream.

Syntax:

Stream<T> sorted()

Stream<T> sorted (Comparator<? super T> comparator)

Example:

```

public class Sorted {
    public static void main (String[] args) {
        List<Integer> list = Arrays.asList (1, 3, 5, 6, 2, 4);
        List<Integer> ascending = list.stream().sorted().collect (Collectors.toList());
        System.out.println (ascending ); // [1, 2, 3, 4, 5, 6]
        List<Integer> descending = list.stream().sorted ( (a, b) -> b-a ).collect (Collectors.toList());
        System.out.println ( descending ); // [6, 5, 4, 3, 2, 1]
    }
}

```

⑤ limit(): The limit method is used to provide the number of elements that we want

Example:

```

public class Limit {
    public static void main (String[] args) {
        List<Integer> list = Arrays.asList (1, 2, 3, 4, 5);
        List<Integer> limit = list.stream().limit (4).collect (Collectors.toList());
        System.out.println ( limit ); // [1, 2, 3, 4]
    }
}

```

⑥ skip(): The skip method will skip the specified number of elements from start

```

public class Skip {
    public static void main (String[] args) {
        List<Integer> list = Arrays.asList (1, 2, 3, 4, 5);
        List<Integer> skip = list.stream().skip (1).collect (Collectors.toList());
        System.out.println ( skip ); // [2, 3, 4, 5]
    }
}

```


⑦ `forEach()`: The `forEach` method is used to Iterate through every element of the stream.

Syntax: `void forEach(Consumer<? Super T> action)`

Example:

```
Public class ForEach {
    Public Static void main (String[] args) {
        List<Integer> list = Arrays.asList(1,2,3,4,5);
        list.stream().forEach(x → System.out.println(x));
    }
}
```

OUTPUT:

1
2
3
4
5

⑧ `reduce()`: The `reduce` method is used to reduce the elements of a stream to a single value.

Example:

```
Public class Reduce {
    Public Static void main (String[] args) {
        List<Integer> list = Arrays.asList(1,2,3,4,5);
        int sum = list.stream().reduce(0, (a,b) → a+b);
        System.out.println(sum); //15
    }
}
```

Explanation:

0 → initial value

$(a,b) \rightarrow a+b \rightarrow$ adds elements step by step

- ① LocalDate: Represents a date without a time zone.

Example:

```

Public class Test {
    Public static void main (String[] args){
        LocalDate now = LocalDate.now(); // Used to find current date
        LocalDate customDate = LocalDate.of (2001, 9, 15);
        int dayOfMonth = now.getDayOfMonth();
        Month month = now.getMonth();
        int year = now.getYear();
        LocalDate yesterday = today now.minusDays (1);
    }
}

```

- ② LocalTime: Represents a time without a date or time zone.

Example:

```

Public class Test {
    Public static void main (String[] args){
        LocalTime now = LocalTime.now();
        LocalTime customTime = LocalTime.of (14, 30, 30);
        System.out.println (customTime ); // 14:30:30
        String timeInString = "15:30:45";
        LocalTime parsedTime = LocalTime.parse (timeInString);
        LocalTime beforeOneHour = now.minusHours (1);
        System.out.println ( beforeOneHour );
    }
}

```


③ **LocalDateTime**: Represents date and time without a time zone.

Example:

```

Public class Test {
    Public Static void main (String[] args) {
        LocalDateTime now = LocalDateTime.now ();
        System.out.println (now ); // 2026-03-27T21:13:30

        LocalDateTime customDateTime = LocalDateTime.of (2026, 03, 27, 21, 40, 30);
        System.out.println (customDateTime ); // 2026-03-27T21:40:30
    }
}

```

④ **ZonedDateTime**: Represents a date and time with a time zone.

Example:

```

Public class Test {
    Public Static void main (String[] args) {
        ZonedDateTime now = ZonedDateTime.now ();
        System.out.println (now ); // 2026-03-27T21:22:45.419+05:30[Asia/Calcutta]

        ZonedDateTime custom = ZonedDateTime.of (2026, 03, 27, 21, 40, 30, ZoneId.of ("America/
        System.out.println (custom ); // 2026-03-27T21:40:30-04:00[America/New-York]
        ZoneId zone = now.getZone ();
    }
}

```

⑤ **Instant**: Represents an instantaneous point on the timeline, typically used for machine timestamps.

Example:

```

Public class Test {
    Public Static void main (String[] args) {
        Instant now = Instant.now ();
        System.out.println (now); // 2026-03-27T23:29.448Z
    }
}

```

⑥ Duration: Represents a duration of time between two points in time.

Example:

```
Public class test {  
    Public static void main (String[] args){  
        Instant start = Instant.now();  
        Instant end = Instant.now();  
        Duration d = Duration.between(start, end);  
    }  
}
```

⑦ Period: Represents a period of time between two dates.

Example:

```
Public class {  
    Public static void main (String[] args){  
        LocalDate now = LocalDate.now();  
        LocalDate then = LocalDate.of(1990, 2, 2);  
        Period period = Period.between(now, then);  
    }  
}
```

⑧ DateTimeFormatter: Formats and Parses date and times.

Example:

```
Public class Test {  
    Public static void main (String[] args){  
        String date = "25/04/1998";  
        DateTimeFormatter format = DateTimeFormatter.ofPattern("dd/MM/yyyy");  
        LocalDate parse = LocalDate.parse(date, format);  
        System.out.println(parse); // 1998-04-25  
    }  
}
```


Optional Class:

Optional class in Java belongs to java.util Package to handle problem of null values more gracefully.

Example: For valid output

```
public class OptionalExample {
    private static Optional<String> getName(int id) {
        String name = "Ram";
        return Optional.of(name);
    }

    public static void main(String[] args) {
        Optional<String> name = getName(2);
        if (name.isPresent()) {
            System.out.println(name.get()); // Ram
        }
    }
}
```

Example: For null values

```
public class NullExample {
    public static Optional<String> getName(int id) {
        String name = null;
        return Optional.ofNullable(name);
    }

    public static void main(String[] args) {
        Optional<String> name = getName(2);
        name.ifPresent(System.out::println); // Null
    }
}
```

Example 3: If we want Default Name, when null value present.

```
public class ORElseMethod {  
    public static Optional<String> getName(int id) {  
        return Optional.empty();  
    }  
    public static void main(String[] args) {  
        Optional<String> name = getName(2);  
        String nameToBeUsed = name.orElse("NA");  
        System.out.println(nameToBeUsed);  
    }  
}
```

Example 4: Throw Exception For Null values

```
public class Exception {  
    public static Optional<String> getName(int id) {  
        return Optional.empty();  
    }  
    public static void main(String[] args) {  
        Optional<String> name = getName(2);  
        String nameToBeUsed = name.orElseThrow(new NoSuchElementException());  
        System.out.println(nameToBeUsed);  
    }  
}
```